Assignment #1

Testing Exception-Handling Policies in Concurrent Systems

A Report

Presented to

The Department of Computer Engineering

Concordia University

In Fulfillment

of the Requirements

of COEN 448

by

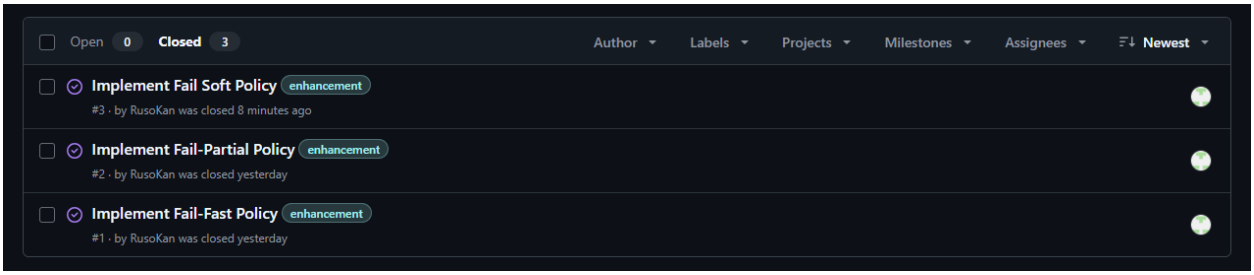
Ruso Kanapathipillai ID: 40133397

Concordia University

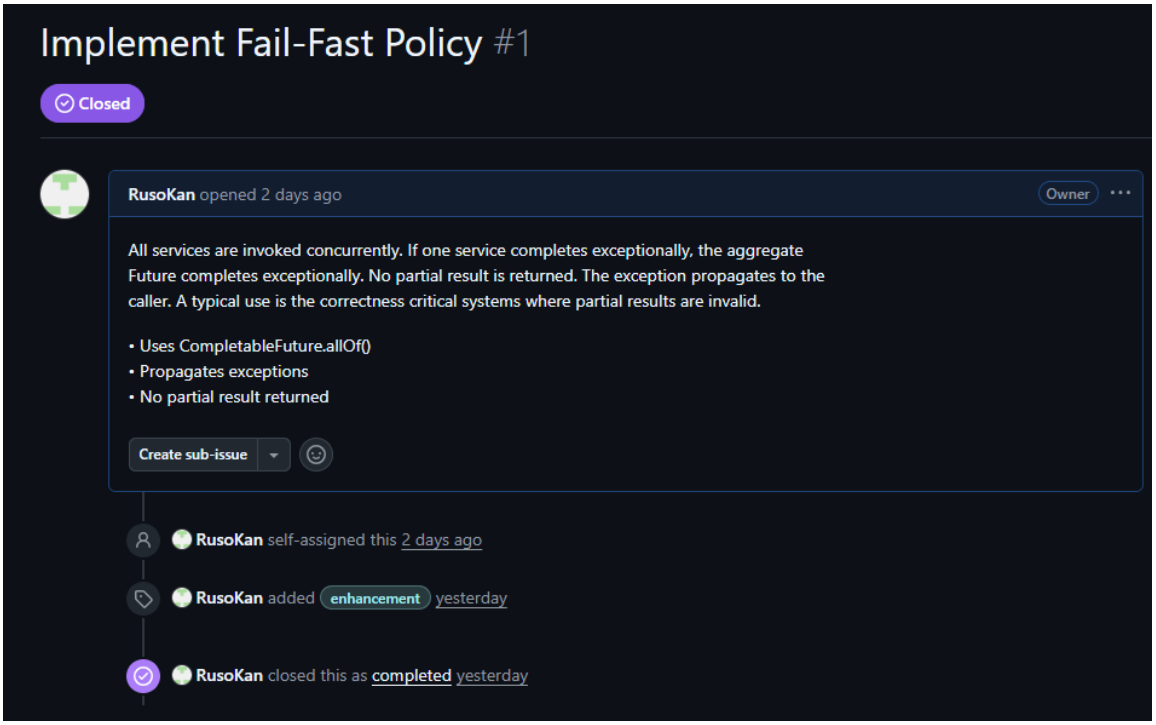
February 16th, 2026

Github Account : [RusoKan/COEN448-Assignment-1]

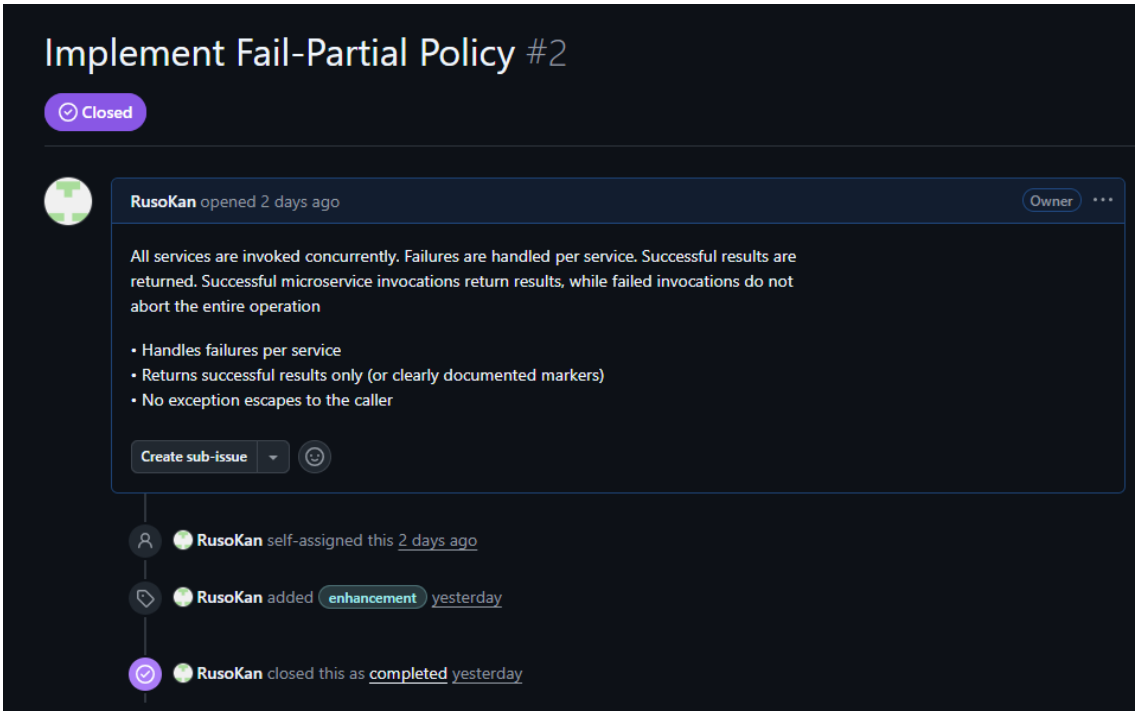
Github Issues



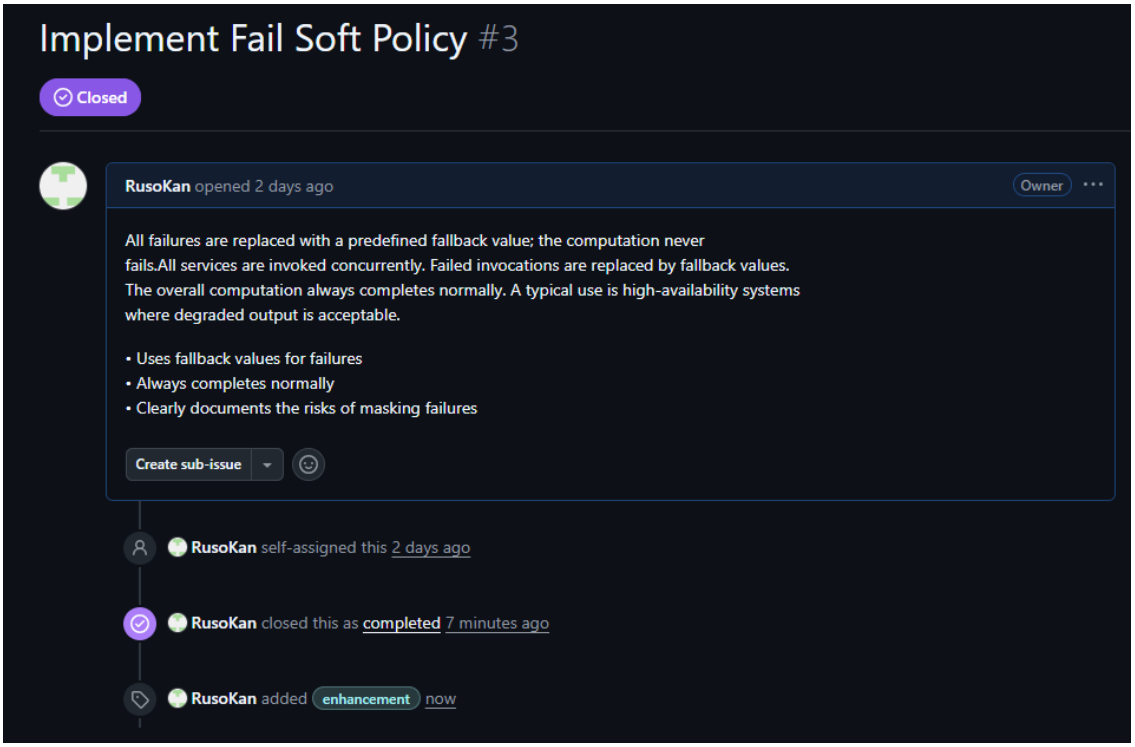
ISSUE #1



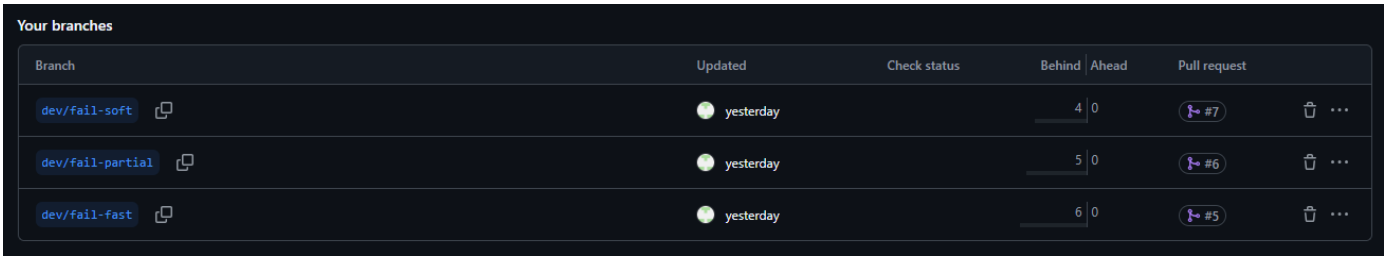
ISSUE #2


Implement Fail-Partial Policy #2
Closed
RusoKan opened 2 days ago
Owner
All services are invoked concurrently. Failures are handled per service. Successful results are returned. Successful microservice invocations return results, while failed invocations do not abort the entire operation
• Handles failures per service
• Returns successful results only (or clearly documented markers)
• No exception escapes to the caller
Create sub-issue
RusoKan self-assigned this 2 days ago
RusoKan added enhancement yesterday
RusoKan closed this as completed yesterday

ISSUE #3

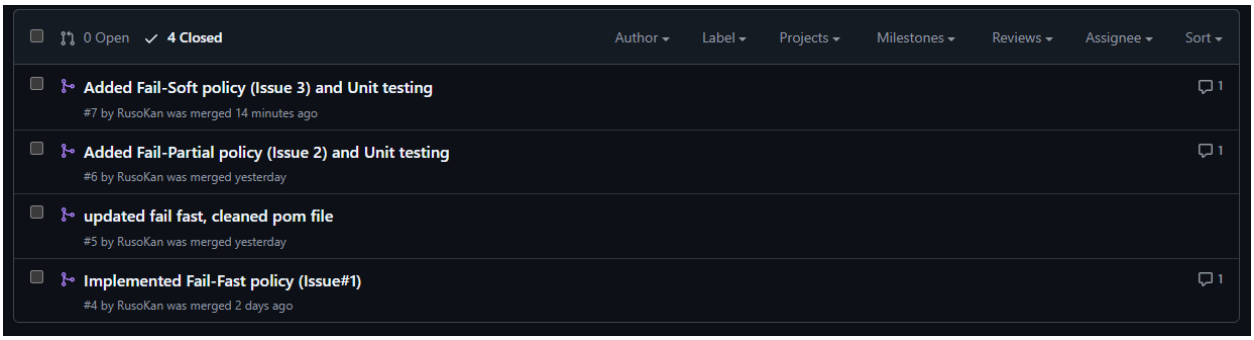

Implement Fail Soft Policy #3
Closed
RusoKan opened 2 days ago
Owner
All failures are replaced with a predefined fallback value; the computation never fails.All services are invoked concurrently. Failed invocations are replaced by fallback values. The overall computation always completes normally. A typical use is high-availability systems where degraded output is acceptable.
• Uses fallback values for failures
• Always completes normally
• Clearly documents the risks of masking failures
Create sub-issue
RusoKan self-assigned this 2 days ago
RusoKan closed this as completed 7 minutes ago
RusoKan added enhancement now

Feature branches

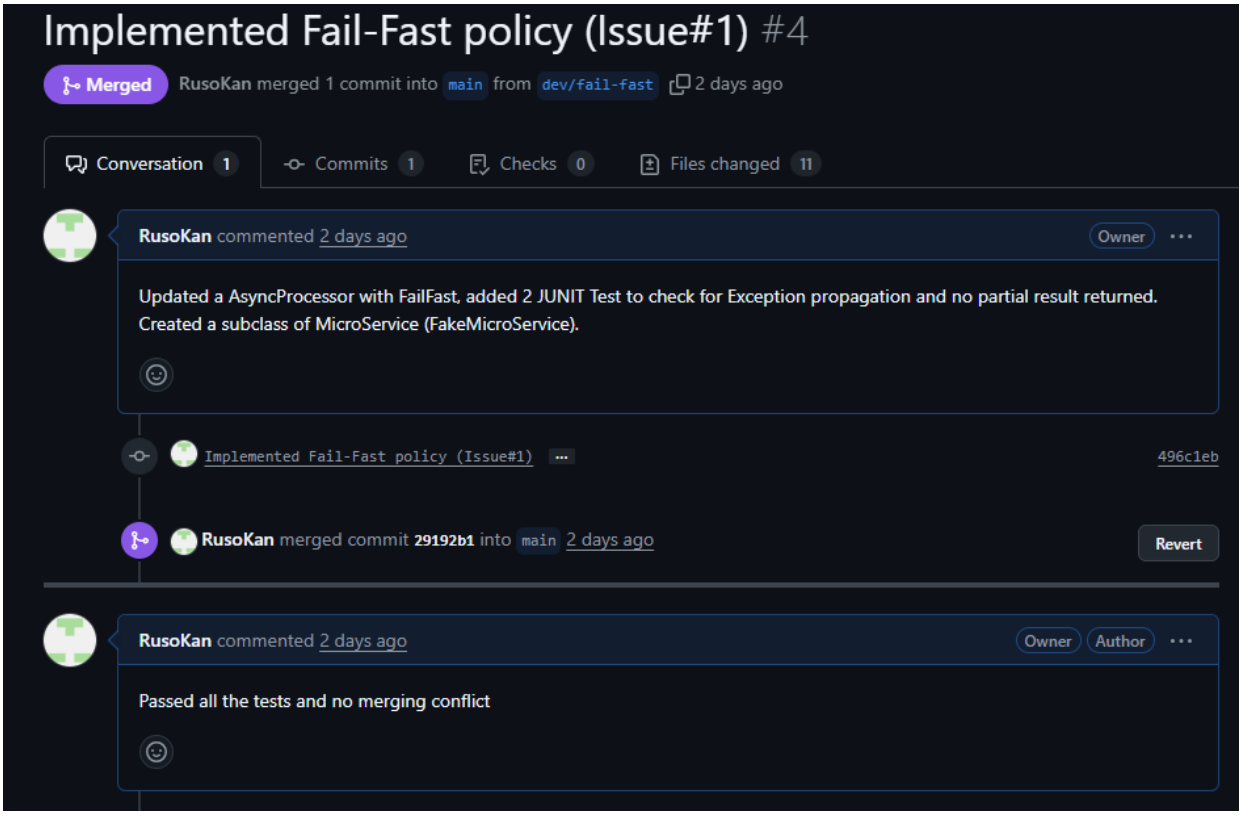


1. dev/fail-soft
2. dev/fail-partial
3. dev/fail-fast

Pull Requests (Summary)

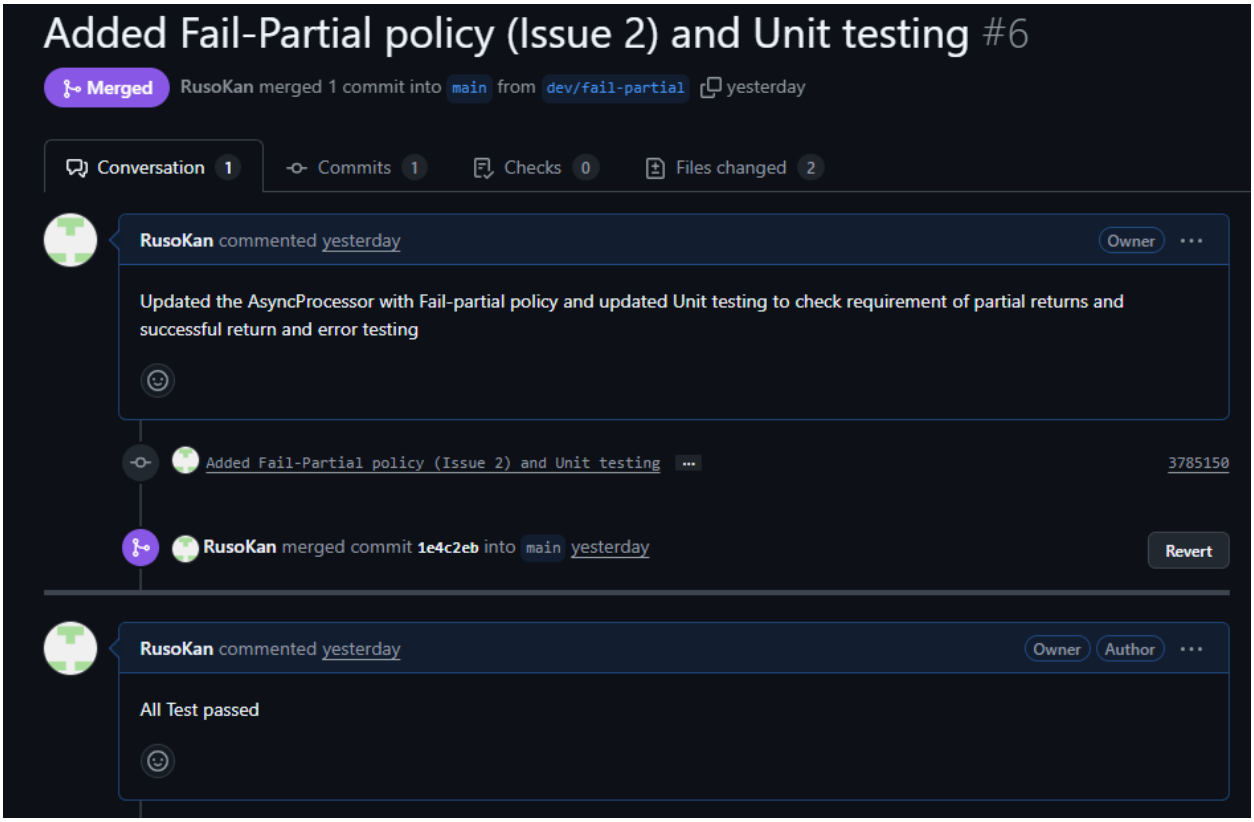


Pull Request #1 from dev/fail-fast



Pull Request #2 from dev/fail-fast

Pull Request #3 from dev/fail-partial



Pull Request # 1 and Peer Review

Given that I was the only one working on this assignment , I self “peer reviewed"

Added Fail-Soft policy (Issue 3) and Unit testing #7

Merged RusoKan merged 1 commit into `main` from `dev/fail-soft` 33 minutes ago

Conversation 2 Commits 1 Checks 0 Files changed 2

RusoKan commented yesterday Owner

Updated the AsyncProcessor with Fail-Soft policy and updated Unit testing to check requirement of successful return of value and return user chosen default value in cases of failure

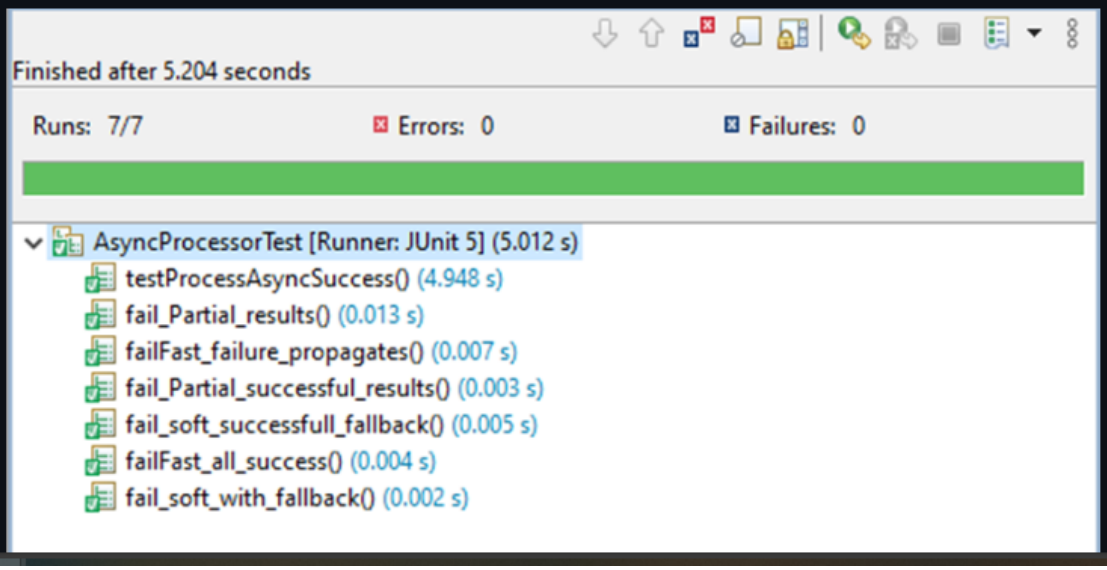
`Added Fail-Soft policy (Issue 3) and Unit testing` ba836c0

RusoKan merged commit `ee40101` into `main` 33 minutes ago Revert

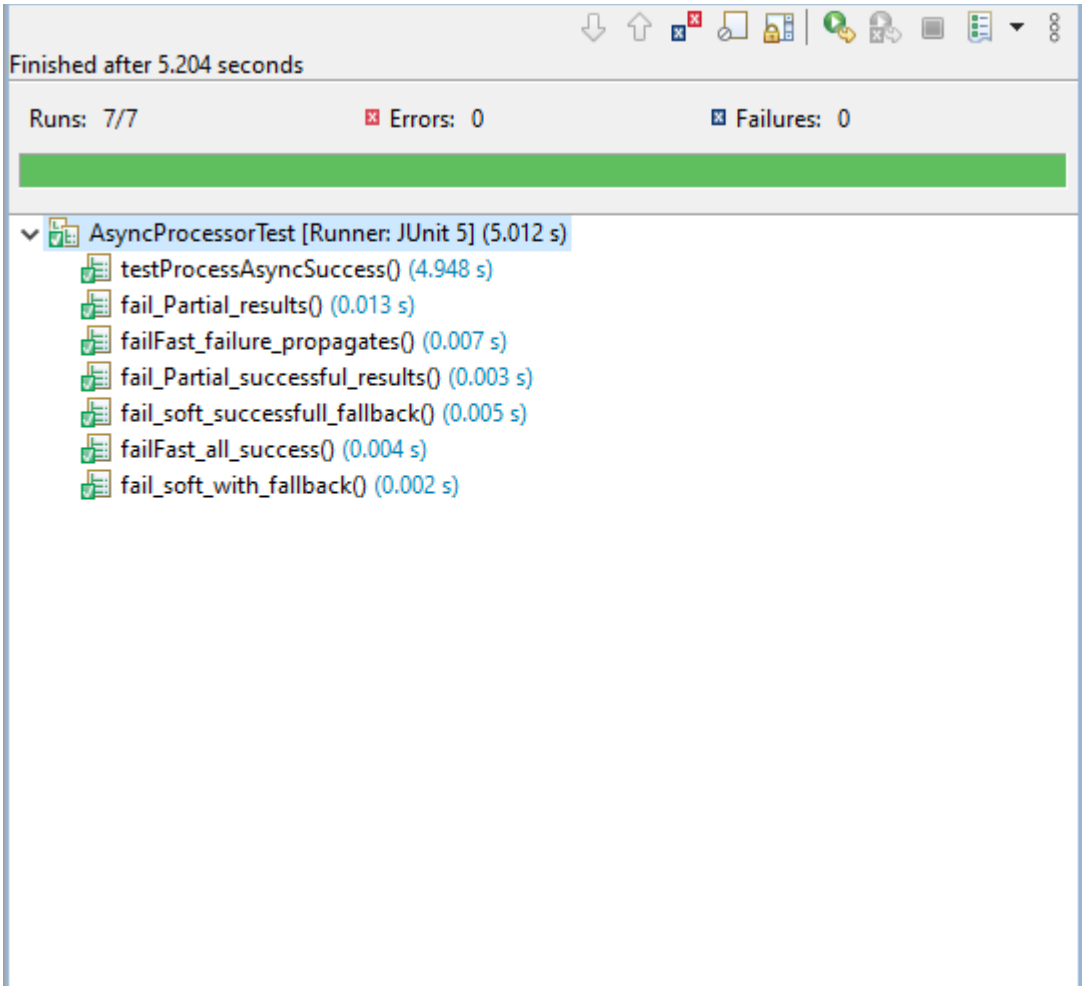
RusoKan commented 1 minute ago View reviewed changes

RusoKan left a comment Owner Author

"Peer Review by QA" : Test have all been confirmed to passed , from issues 1,2,3 .



UNIT TESTING Screenshots



Explanation of JUnit tests:

```
public void failFast_failure_propagates()
```

This Test Validates the 'Fail Fast' mechanism by ensuring that when a single microservice failure happens , it immediately triggers an exception, preventing any partial data from passing within a reasonable timeout. In this case , two microservices were given, Microservice s1 was simulated to fail, and microservice s2 to pass, `assertThrows(ExecutionException.class, () ->`

`result.get(2, TimeUnit.SECONDS));` This checks that the results is an exception and failure is propagated so no partial result, s2 Is returned. `assertTrue(result.isCompletedExceptionally());` this also assert that result has been thrown an exception

`public void failFast_all_success()`

This test checking that Test passes succesfully for Fail Fast when all microservices passes to make sure that it is healthy and does not give wrong result. This is a positive test for a comprehensive testing

`public void fail_Partial_results()`

This test verifies the 'Partial Success' pattern by ensuring that the processor captures individual service failures as specific error strings within the result list, allowing the user to receive data from healthy services while identifying exactly which ones failed. In this scenario, we have two microservices s1 and s2 , s1 is simulated to fail , while s2 is simulated to pass , as we pass those results to the AsyncProcessor, we expected the following:

```
//Partial result can be returned, since s1 Microservice failed result returns failed!
        List<String> expected = Arrays.asList("Failed!", "MESSAGE2");
       //checks that the list matches the result fails within a timeout of 2 seconds
        assertEquals(expected,  result.get(2, TimeUnit.SECONDS));
```

The Assert false `assertFalse(result.isCompletedExceptionally());` for exceptions was also included, since the failed! is marked as instructed in the assignment1.

`public void fail_Partial_successful_results()`

This test Validates the data consistency of the partial failure mechanism, confirming that when no errors occur, the processor correctly transforms and collects all successful microservice responses. This is a positive Testing for a comprehensive testing.

`public void fail_soft_with_fallback()`

This test verifies that when a microservice fails, the processor replaces the missing data with a user defined fallback value making sure the execution always completes normally within a reasonable timeout. In this case, 3 Microservices were put into the processor, and Microservices s2 is simulated to fail, and a fallback of default was given. `assertEquals("MESSAGE1 default MESSAGE3", result.get(2, TimeUnit.SECONDS));` was asserted, and test passed. Also, since failure was handled, `assertFalse(result.isCompletedExceptionally());` assert False helps check to make sure no exceptions was given.

`public void fail_soft_successfull_fallback()`

This Test validates to ensure that when all microservices are healthy, the processor correctly aggregates the full results without triggering the fallback logic or giving the wrong results. Positive Test Case for a comprehensive testing.

USE OF AI TRANSPARENCY

The Use of AI Tool that was used was ChatGPT (OpenAI) to get some of the initial boilerplate and to assist with understanding concepts that were not clear, and the instructions in the creation of a maven project, improvements on the code.

The AI was for example, used to create a subclass of MicroService (FakeMicroService) using the following prompt to get the initial boilerplate:

"Write a Java class named FakeMicroService that extends a Microservice class. It should have a boolean field shouldFail set via the constructor. Override a method retrieveAsync(String input) that returns a CompletableFuture<String>. If shouldFail is true, return a future completed exceptionally with a run time exception, otherwise, return a completed future containing the uppercase version of the input."